

【专栏二：深度学习03】 - 【一张图讲清楚：为什么神经网络一定要有激活函数？“非线性”到底重要在哪？】



快乐飞猪ing



· 码龄7年 · 粉丝可见 · 编辑



墨衍会员 · AI 创作全网分发

原创 · 2026-03-27 12:01:45 · 664 阅读



AI 写同款 · GEO 优化 >

非线性激活函数 (Activation Function)

[阅读详情 >](#)

所谓激活函数 (Activation Function) , 就是在人工神经网络的神经元上运行的函数, 负责将神经元的输...

前言

上一篇文章里, 我讲了深度学习 训练过程中的向前传播和向后传播。但继续往下学, 一个问题迟早会出现:

神经网络为什么一定要有激活函数?

因为从表面上看, 每一层都在做类似 $z = Wx + b$ 的计算。

那问题来了:

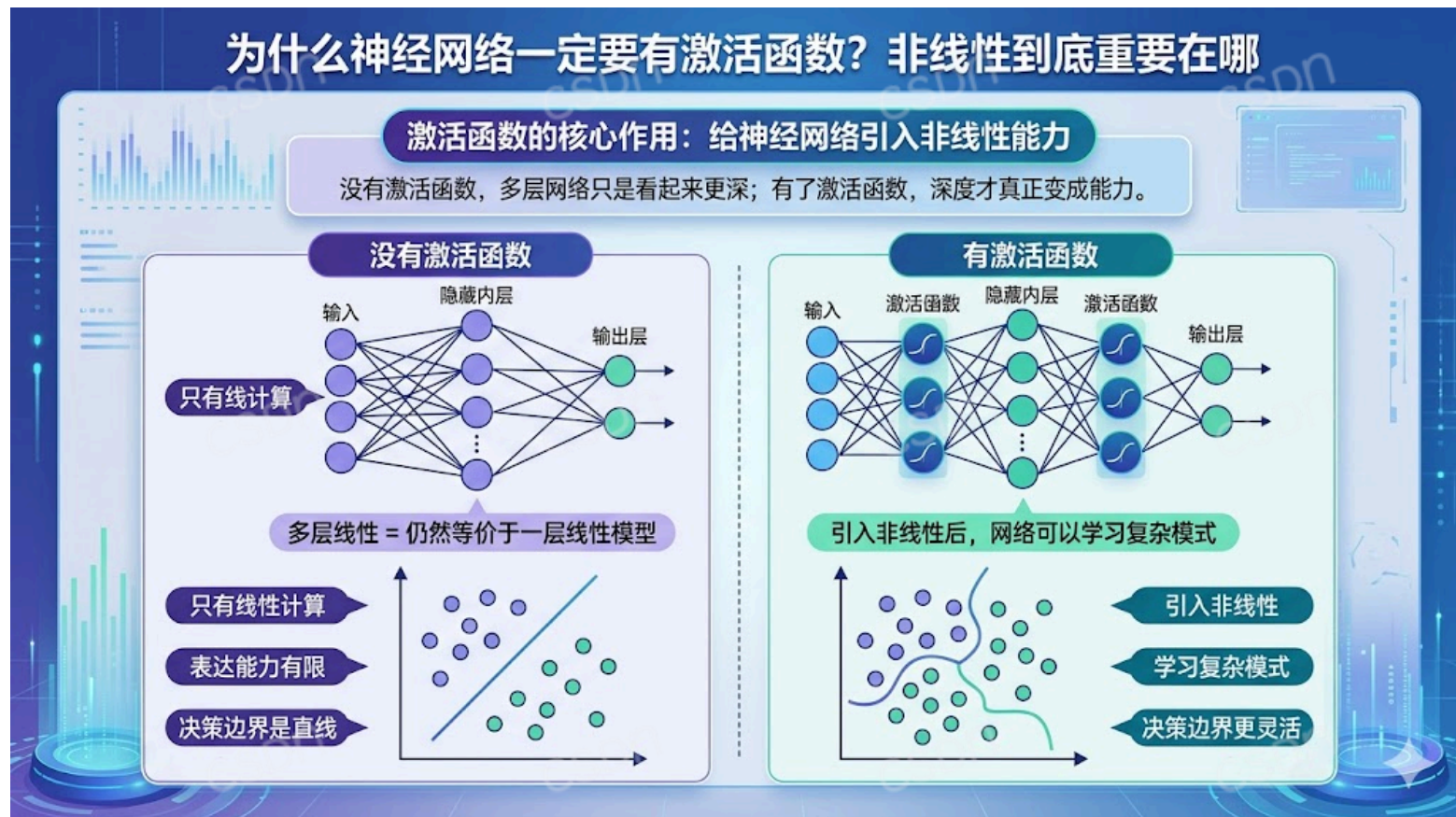
- 如果每一层都只是线性计算, 多堆几层不就行了吗?
- 为什么还必须额外加入激活函数?
- 这里的“非线性”到底重要在哪?

内容来源: csdn.net

作者昵称: 快乐飞猪ing

原文链接: https://blog.csdn.net/m0_46279060/article/details/159515951

作者主页: https://blog.csdn.net/m0_46279060



先说结论：

激活函数最核心的作用，不是“多一个函数”，而是给神经网络引入非线性能力。

如果没有**激活函数**，那么无论网络堆多少层，本质上都还是一个线性模型。

也就是说：没有激活函数，多层网络只是“看起来更深”；**有了激活函数，深度才真正变成能力。**

一、没有激活函数，多层网络为什么还是不够？

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159515951

作者主页：https://blog.csdn.net/m0_46279060

先看图左边，它表示的是：**没有激活函数的神经网络**。

结构上看，它也有：

- 输入层
- 隐藏层
- 输出层

但问题在于：层与层之间只有**线性**  计算。

也就是说，每一层都在做： $z=wx+b$

这会带来三个直接后果：

- 只有线性计算
- 表达能力有限
- 决策边界只能是直线

看起来层数很多，但本质上能力并没有真正提升。

二、关键问题：多层线性，真的更强吗？

很多人直觉会觉得：

- **一层不够 → 堆两层**

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159515951

作者主页：https://blog.csdn.net/m0_46279060

- 两层不够 → 堆三层
- 层数越多 → 能力越强

但这里有一个非常反直觉的结论：

如果每一层都是线性计算，那么无论堆多少层，最后仍然等价于一层线性模型 [🔗](#)。

1、数学证明：多层线性=一层线性

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159515951

作者主页：https://blog.csdn.net/m0_46279060

假设一个两层网络：

第一层：

$$h = W_1x + b_1$$

第二层：

$$y = W_2h + b_2$$

把第一层代入第二层：

$$y = W_2(W_1x + b_1) + b_2$$

展开：

$$y = W_2W_1x + W_2b_1 + b_2$$

我们重新定义：

$$W' = W_2W_1$$

$$b' = W_2b_1 + b_2$$

那么：

$$y = W'x + b'$$

👉 这说明什么？

两层线性变换，可以完全合并成一层新的线性变换。

那三层、四层呢？

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159515951

作者主页：https://blog.csdn.net/m0_46279060

结论完全一样。

只要没有激活函数，不管你堆多少层，最终都可以写成：

$$y = Wx + b$$

所以图左边那句话是一个严格的数学事实：

多层线性网络，本质仍然是线性模型。

2、伪代码解释

我们用伪代码再看一遍。

✗ 没有激活函数

```
c AI写代码 | 运行
1 x = input_data
2
3 h1 = W1 @ x + b1
4 h2 = W2 @ h1 + b2
5 y = W3 @ h2 + b3
```

看起来是三层网络，但本质上，它可以被压缩成： $y = W @ x + b$

👉 层数只是“写多了几行代码”，并没有带来本质能力提升。

三、有了激活函数，真正的变化发生了

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159515951

作者主页：https://blog.csdn.net/m0_46279060

1、数学证明：

现在看图右边。

唯一的变化是：

层与层之间加入了激活函数。

也就是说，网络结构变成了：

$$h = g(W_1x + b_1)$$

$$y = W_2h + b_2$$

这里多出来的 $g(\cdot)$ ，就是激活函数。

2、伪代码类比

✗ 不加激活函数

```
go AI写代码 | 运行  
1 | x = input_data  
2 | h1 = W1 @ x + b1  
3 | h2 = W2 @ h1 + b2  
4 | y = W3 @ h2 + b3
```

✓ 加入激活函数

```
go AI写代码 | 运行  
1 | x = input_data  
2 | h1 = relu(W1 @ x + b1)
```

内容来源: csdn.net

作者昵称: 快乐AI写代码

原文链接: https://blog.csdn.net/m0_46279060/article/details/159515951

作者主页: https://blog.csdn.net/m0_46279060

```
3 | h2 = relu(w2 @ h1 + b2)
4 | y = sigmoid(w3 @ h2 + b3)
```

区别只在一件事：

中间多了非线性函数 (relu / sigmoid)

但正是这个变化，让网络能力发生质变。

四、！重点 为什么现实问题一定需要“非线性”？

回到现实任务，如果数据很简单，比如：👉 两类点可以用一条直线分开

那线性模型就够了，但现实世界不是这样：

- 图片里的猫 vs 非猫
- 用户行为模式
- 语音、文本

这些问题的本质都是：**复杂、非线性、高维结构**，如果模型只能画“直线”，那它永远学不会这些规律。而**激活函数带来的非线性能力**，让模型可以：

- 学习弯曲的决策边界
- 组合复杂特征
- 一层层抽象信息

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159515951

作者主页：https://blog.csdn.net/m0_46279060

这就是为什么图右边的边界是“曲线”，而不是“直线”。

1、什么叫“两类点可以用一条直线分开”？

先别想猫、语音、文本，先想最简单的二维平面。

假设有两类数据点：

- 蓝色点
- 绿色点

如果它们长这样：

- 蓝色点都在左边
- 绿色点都在右边

那你只要画一条直线，就能把它们分开。

比如：

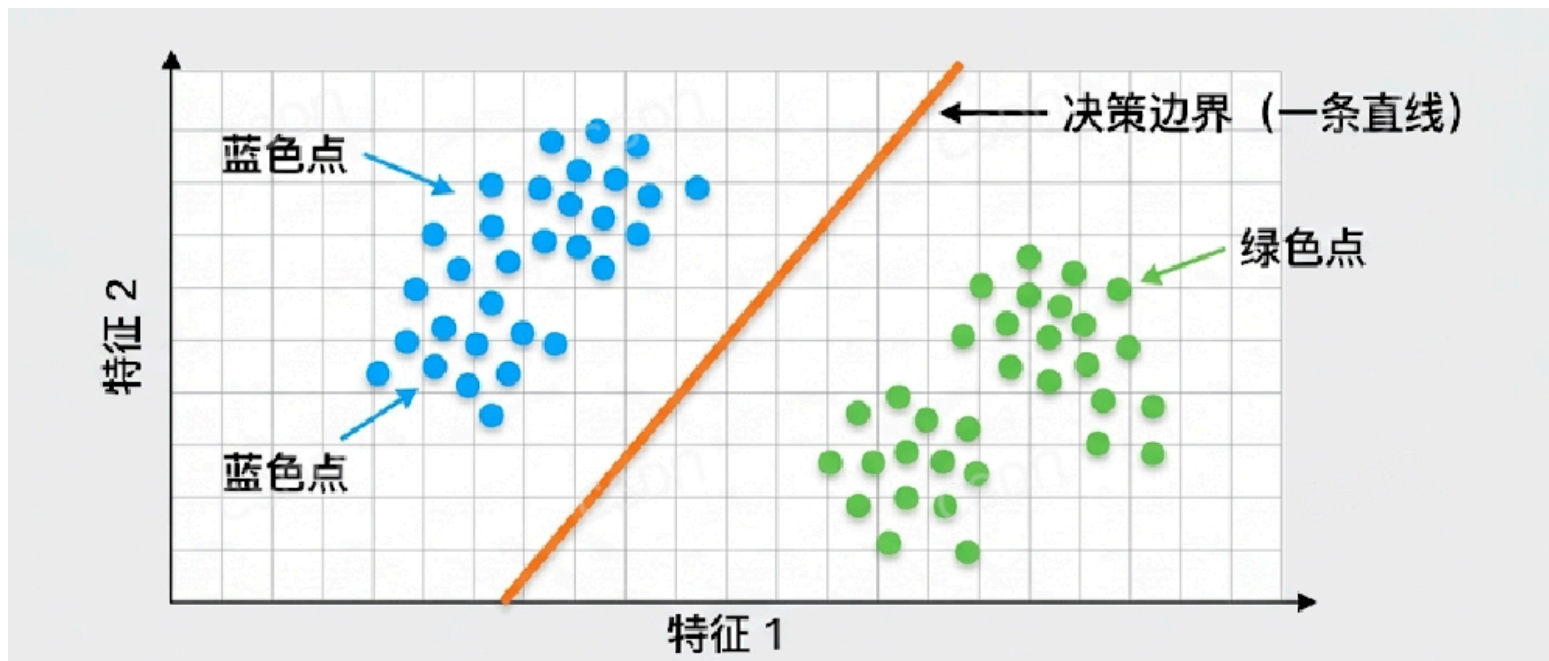
蓝 蓝 蓝 | 绿 绿 绿

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159515951

作者主页：https://blog.csdn.net/m0_46279060



这时候，一条线就够了，这类问题就叫：**线性可分问题**，
也就是：**数据可以被一条直线（二维）、一个平面（三维）或一个超平面（更高维）分开。**

2、什么叫“线性模型就够了”

线性模型本质上做的事情很简单：**根据输入特征，算一个线性分数，再决定属于哪一类。**

比如最常见的形式： $y=wx+b$

它的意思你可以理解成：

- 把输入特征乘上权重
- 加起来

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159515951

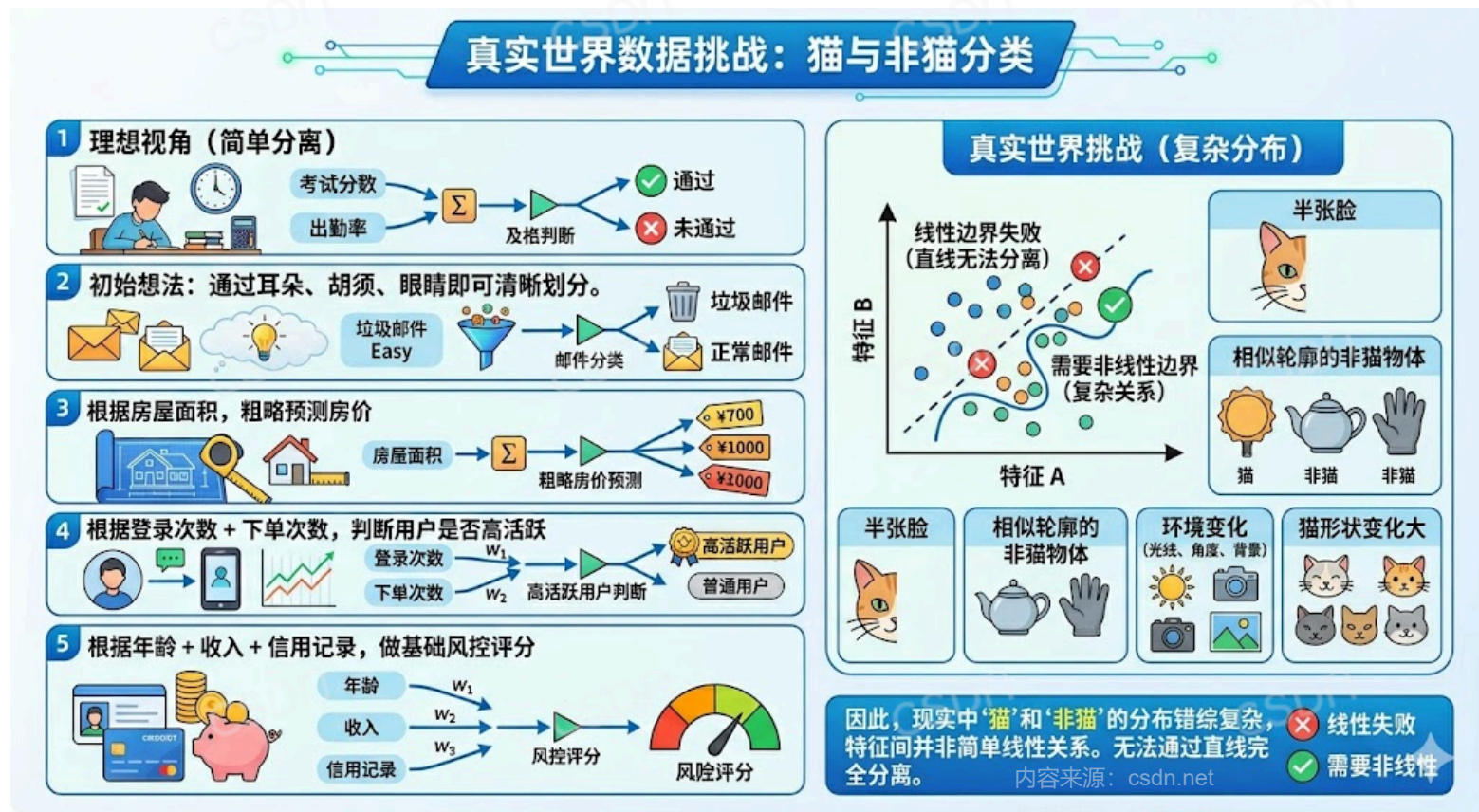
作者主页：https://blog.csdn.net/m0_46279060

- 再加个偏置
- 最后看结果落在哪边

这个“落在哪边”，其实就对应一条分界线。

所以线性模型最擅长的，就是：用一条直线或者一个平面来切分类别。

如果你的数据本来就能被一条直线分开，那它就很好用。



3、为什么现实世界的问题不是这样

现在回到真实任务，比如“猫 vs 非猫”。

你可能会想：

- 猫有耳朵
- 猫有胡须
- 猫有眼睛

但问题是，现实中的图片并不会整整齐齐排好，让你一条线就分开：

- 有些猫只露半张脸
- 有些非猫物体也可能有相似轮廓
- 光线、角度、背景都不一样
- 猫的形状变化也很大

所以“猫”和“非猫”的分布，通常不是这种简单情况：

猫 猫 猫 | 非猫 非猫 非猫

猫 猫 猫 | 非猫 非猫 非猫

更可能是：

一部分猫在这里

一部分非猫在那边

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159515951

作者主页：https://blog.csdn.net/m0_46279060

有的区域还互相交叉

特征之间的关系也不是简单加法

这时候，你就没法靠“一条直线”把它们干净分开。

4、什么叫“复杂、非线性、高维结构”

这三个词其实是在形容真实数据的难度。

(1) 复杂

意思是：**规律不是一句简单规则就能说清的。**

比如猫的判断，不是“有耳朵就是猫”这么简单。

因为狗也有耳朵，兔子也有耳朵。

真正的判断往往是：

- 耳朵形状
- 眼睛位置
- 脸部比例
- 胡须纹理
- 整体轮廓

这些东西要组合起来看，所以叫“复杂”。

(2) 非线性

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159515951

作者主页：https://blog.csdn.net/m0_46279060

“非线性”你可以先理解成：**输入和结果之间的关系，不是一条直线能表达的。**

比如你只有两个特征：

- **x1**：耳朵尖不尖
- **x2**：胡须明显不明显

如果这两个特征和“是不是猫”的关系很复杂，比如要弯着分、绕着分、组合着分，那这就是非线性关系，也就是说：

- 不是 x1 越大就一定越像猫
- 不是 x2 越大就一定越像猫
- 而是它们某种**组合模式**才重要

这种关系，线性模型很难表达。

(3) 高维结构

这是说：

真实问题通常不是只有 2 个特征，而是有很多很多特征。

比如一张图片，可能有成千上万个**像素 Q**点。

即使经过**特征提取 Q**，也可能有很多维度。

所以真实任务不是二维平面画点那么简单，而是在一个非常高维的空间里做分类。

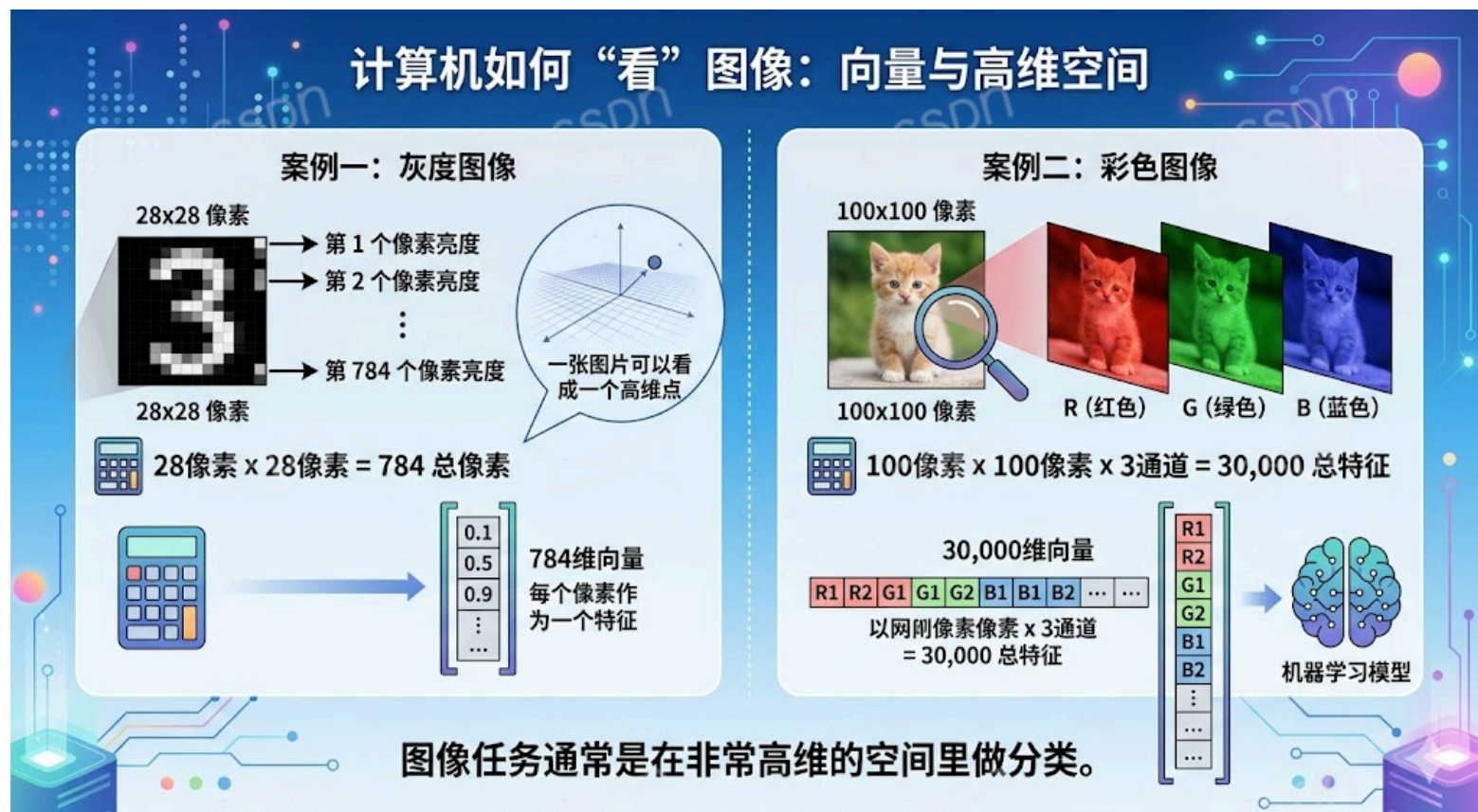
内容来源：csdn.net

作者昵称：快乐飞猪ing

链接：https://blog.csdn.net/m0_46279060/article/details/159515951

作者主页：https://blog.csdn.net/m0_46279060

你虽然看不到这个空间，但模型是在那个空间里学习规律。



5、为什么“如果模型只能画直线，它永远学不会这些规律”

“画直线”其实就是在比喻：模型只能做线性划分。

如果模型只能线性划分，那它就只能处理这种数据：

- A 类在一边
- B 类在另一边
- 一刀切开

内容来源: csdn.net

作者昵称: 快乐飞猪ing

原文链接: https://blog.csdn.net/m0_46279060/article/details/159515951

作者主页: https://blog.csdn.net/m0_46279060

但现实数据常常需要的是：

- 弯着切
- 分段切
- 多层组合后再切

所以如果模型只能线性，它面对复杂任务时就会出现两个问题：

1. 分不开

不管怎么调参 Q 数，直线就是分不开。

2. 表达不了

数据背后的真实关系，它根本没有能力表示出来。

这就是为什么只靠线性层，多层堆再多也没用。

内容来源：csdn.net

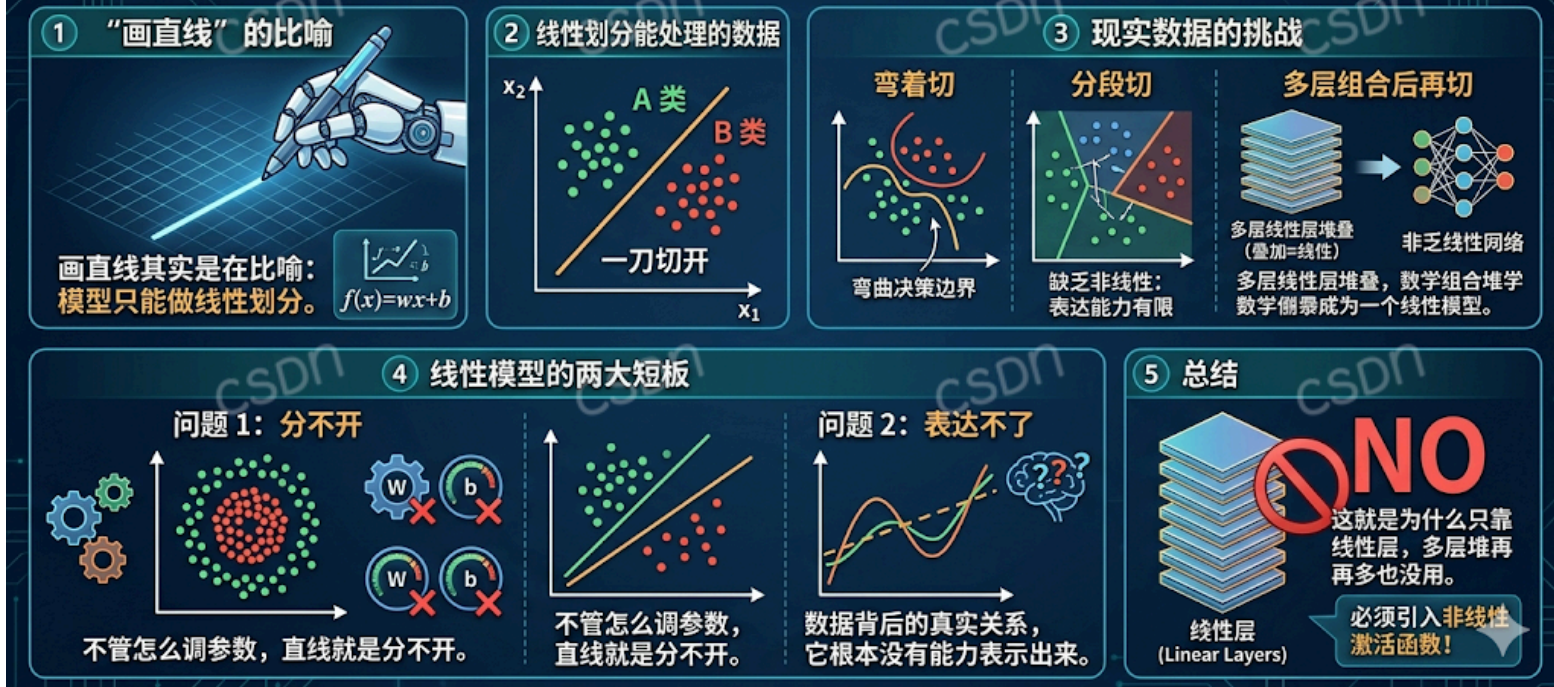
作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159515951

作者主页：https://blog.csdn.net/m0_46279060

V. 为什么“如果模型只能画直线，它永远学不会这些规律”

深入理解线性模型的局限性



6、什么叫“学习弯曲的决策边界”

“决策边界”你可以理解成：模型用来区分两类数据的分界线。

如果是线性模型，这条边界通常就是直线。

如果加入非线性，这条边界就可以变成：

- 曲线
- 弯折的线

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159515951

作者主页：https://blog.csdn.net/m0_46279060

- 更复杂的形状

为什么这很重要？因为真实数据分布常常就是需要这种复杂边界才能分开。

所以这句话：**学习弯曲的决策边界**

本质上就是在说：

模型可以不再局限于“直线分割”，而能用更灵活的方式去区分类别。

7、什么叫“组合复杂特征”

这个特别适合用“猫”的例子理解。

模型一开始看到的，可能只是一些很基础的模式：

- 边缘
- 纹理
- 小块形状

这些都还不是“猫”。但随着层数变深，模型会逐渐组合它们：

- 边缘 + 边缘 → 局部轮廓
- 局部轮廓 + 纹理 → 耳朵/眼睛/胡须区域
- 多个局部结构再组合 → 猫脸、猫身体、整体姿态

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159515951

作者主页：https://blog.csdn.net/m0_46279060

这就叫：**组合复杂特征**

如果没有非线性，这种“逐层组合、逐层抽象”的能力会非常受限。

8、什么叫“一层层抽象信息”

这句话的意思是：

模型前面几层学简单的东西，后面几层学更复杂、更抽象的东西。

比如在图像任务里：

- 前层：边缘、亮暗、线条
- 中层：角、纹理、局部形状
- 后层：耳朵、眼睛、面部轮廓
- 更后层：完整的猫

所以“一层层抽象”其实就是：

- 从低级特征走向高级特征
- 从局部走向整体
- 从具体像素走向语义概念

而这个过程的前提，就是网络必须有非线性表达能力。

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159515951

作者主页：https://blog.csdn.net/m0_46279060

CSDN

CSDN

CSDN

CSDN

CSDN

CSDN

CSDN

CSDN

CSDN

CSDN

CSDN

CSDN

CSDN

CSDN

CSDN

CSDN

CSDN

CSDN

CSDN

CSDN

CSDN

CSDN

CSDN

CSDN

内容来源: csdn.net
作者昵称: 快乐飞猪ing
原文链接: https://blog.csdn.net/m0_46279060/article/details/159515951
作者主页: https://blog.csdn.net/m0_46279060